Universidad de Guadalajara
cugdl
Lic. en Inteligencia Artificial y Ciencia de Datos
PROGRAMACIÓN IV
Centro Universitario Guadalajara CUGDL

Refuerzo

FastAPI, Uvicorn y el CRUD de Tareas

Una explicación introductoria y código documentado línea por línea

---

Autor:
Patricia Jaime Pérez

Profesor:
Diego Alberto Ayala Vázquez

Fecha:
16 de septiembre de 2026

16 de septiembre de 2026

Índice

1. ¿Qué es FastAPI y qué es Uvicorn? **2**
1.1. Una analogía: la biblioteca pública 2
1.2. ¿Por qué se requieren ambos componentes? 2
1.3. La diferencia entre ASGI y el antiguo WSGI 2

2. Código documentado línea por línea: CRUD de Tareas **3**

3. Procedimiento de ejecución **7**

4. Resumen del flujo de comunicación **7**

Referencias **8**

1. ¿Qué es FastAPI y qué es Uvicorn?

1.1. Una analogía: la biblioteca pública

Para comprender el papel de FastAPI y Uvicorn puede establecerse una comparación con el funcionamiento de una biblioteca pública.

FastAPI corresponde al bibliotecario. Es quien conoce el catálogo completo, sabe qué libros existen, en qué estantería se encuentran y qué reglas deben seguirse para prestarlos. Cuando un usuario solicita un libro, el bibliotecario verifica que la solicitud sea válida, busca el ejemplar y prepara la respuesta. De manera análoga, FastAPI recibe las peticiones de las aplicaciones externas, valida los datos, ejecuta la lógica correspondiente y construye la respuesta en un formato que el solicitante pueda entender.

Uvicorn corresponde al mostrador de recepción. Es el punto físico por donde entran las solicitudes y salen las respuestas. El mostrador no decide qué libro entregar ni cómo buscarlo; su función consiste en recibir a cada visitante, canalizar su petición hacia el área correspondiente y entregarle después el resultado. De manera análoga, Uvicorn es el componente que escucha las conexiones de red, traduce el lenguaje HTTP al lenguaje interno del servidor y devuelve la respuesta al cliente.

1.2. ¿Por qué se requieren ambos componentes?

La biblioteca no podría funcionar solo con el bibliotecario encerrado en su oficina, sin un mostrador que atendiera al público. Tampoco podría funcionar con un mostrador vacío, sin nadie que supiera localizar los libros. Cada componente cumple una función distinta y complementaria:

- Sin Uvicorn, las peticiones de los clientes nunca llegarían a FastAPI.
- Sin FastAPI, Uvicorn no dispondría de una aplicación capaz de generar respuestas.

La combinación de ambos da como resultado un servicio web completo.

1.3. La diferencia entre ASGI y el antiguo WSGI

El estándar anterior, conocido como WSGI, funcionaba como una biblioteca con una única ventanilla de atención: cada visitante debía ser atendido por completo antes de que el siguiente pudiera acercarse. Si una consulta tomaba mucho tiempo, el resto de los usuarios permanecía esperando en la fila.

ASGI, en cambio, equivale a una biblioteca con varias ventanillas y personal capaz de atender solicitudes en paralelo. Mientras una consulta larga se encuentra en proceso, otras pueden ser atendidas simultáneamente. Esta característica es la que permite que las aplicaciones modernas respondan con mayor rapidez cuando reciben muchas peticiones al mismo tiempo.

2. Código documentado línea por línea: CRUD de Tareas

A continuación se presenta el archivo `main.py` completo con un CRUD (Crear, Leer, Actualizar, Eliminar) de tareas. Cada línea de código incluye un comentario que indica su propósito.

```python
# ============================================================
# main.py - API de Tareas con FastAPI
# CRUD = Create (Crear), Read (Leer), Update (Actualizar), Delete (Eliminar)
# ============================================================

# Se importa FastAPI: clase principal del framework que crea la aplicacion
from fastapi import FastAPI, HTTPException

# Se importa BaseModel de Pydantic: define la "forma" que tendran los datos
# Pydantic valida automaticamente que los datos recibidos cumplan con esa forma
from pydantic import BaseModel

# Se importa Optional de typing: indica que un campo puede tener valor o ser None
from typing import Optional

# Se importa List: permite indicar que una variable es una lista de elementos
from typing import List

# ------------------------------------------------------------
# PASO 1: Se crea la aplicacion FastAPI
# app es el objeto que contendra todas las rutas y la logica de la API
# ------------------------------------------------------------
app = FastAPI(
    title="API de Tareas",                    # Titulo mostrado en la documentacion
    description="Un CRUD simple de tareas",   # Descripcion de la API
    version="1.0.0"                           # Version de la API
)

# ------------------------------------------------------------
# PASO 2: Se define el "modelo" de datos con Pydantic
# Esto indica a FastAPI como deben verse las tareas recibidas y devueltas
# ------------------------------------------------------------
```

```python
class Tarea(BaseModel):
    # id: numero entero que identifica la tarea.
    # Optional[int] = None indica que al crear una tarea nueva,
    # el cliente no envia el id (lo genera el servidor)
    id: Optional[int] = None

    # titulo: texto obligatorio que describe la tarea
    titulo: str

    # descripcion: texto opcional con detalles adicionales de la tarea
    descripcion: Optional[str] = None

    # completada: valor verdadero/falso que indica si la tarea ya se termino.
    # Por defecto es False (no completada)
    completada: bool = False

# ------------------------------------------------------------
# PASO 3: Se crea una "base de datos" en memoria
# En un proyecto real se usaria una base de datos (PostgreSQL, SQLite, etc.)
# Aqui se emplea una lista de Python para simplificar el ejemplo
# ------------------------------------------------------------

# Lista vacia que almacenara todas las tareas mientras el servidor este activo
tareas_db: List[Tarea] = []

# Contador para generar IDs automaticamente (comienza en 1)
contador_id: int = 1

# ------------------------------------------------------------
# PASO 4: Se definen las rutas (endpoints) de la API
# Cada ruta responde a un metodo HTTP especifico
# ------------------------------------------------------------

# --- RUTA 1: Pagina de inicio (solo para verificar que la API responde) ---
@app.get("/")  # GET / responde cuando se visita la raiz del servidor
def raiz():
    # Devuelve un diccionario de Python; FastAPI lo convierte a JSON
    return {"mensaje": "Bienvenido a la API de Tareas"}

# --- RUTA 2: CREAR una nueva tarea (POST) ---
# POST se emplea para crear recursos nuevos
```

```python
# El parametro "tarea: Tarea" indica a FastAPI que el cuerpo de la peticion
# debe coincidir con el modelo Tarea
@app.post("/tareas/", response_model=Tarea, status_code=201)
def crear_tarea(tarea: Tarea):
    # Se declara "global" para modificar la variable contador_id externa
    global contador_id

    # Se asigna el ID generado automaticamente a la nueva tarea
    tarea.id = contador_id

    # Se incrementa el contador para la siguiente tarea
    contador_id += 1

    # Se agrega la tarea a la lista (la "base de datos")
    tareas_db.append(tarea)

    # Se devuelve la tarea creada (con su id asignado)
    return tarea

# --- RUTA 3: LEER todas las tareas (GET) ---
# GET se emplea para obtener informacion sin modificarla
@app.get("/tareas/", response_model=List[Tarea])
def leer_tareas():
    # Se devuelve la lista completa de tareas
    # response_model=List[Tarea] indica que se devuelve una lista de Tarea
    return tareas_db

# --- RUTA 4: LEER una tarea especifica por su ID (GET) ---
# {tarea_id} es un "parametro de ruta": el cliente coloca el numero en la URL
# Ejemplo: GET /tareas/1 devuelve la tarea con id=1
@app.get("/tareas/{tarea_id}", response_model=Tarea)
def leer_tarea(tarea_id: int):
    # Se recorre la lista de tareas
    for tarea in tareas_db:
        # Si se encuentra una tarea con el ID solicitado...
        if tarea.id == tarea_id:
            # ...se devuelve
            return tarea

    # Si el bucle termina sin encontrarla, se lanza un error 404
    # HTTPException es la forma que tiene FastAPI de devolver errores HTTP
    raise HTTPException(status_code=404, detail="Tarea no encontrada")
```

```python

# --- RUTA 5: ACTUALIZAR una tarea existente (PUT) ---
# PUT se emplea para reemplazar completamente un recurso existente
@app.put("/tareas/{tarea_id}", response_model=Tarea)
def actualizar_tarea(tarea_id: int, tarea_actualizada: Tarea):
    # Se recorre la lista con indice (enumerate proporciona posicion y elemento)
    for indice, tarea in enumerate(tareas_db):
        # Si se encuentra la tarea con el ID correcto...
        if tarea.id == tarea_id:
            # Se actualiza el ID de la tarea nueva (por si el cliente no lo envio)
            tarea_actualizada.id = tarea_id

            # Se reemplaza la tarea anterior por la nueva en la misma posicion
            tareas_db[indice] = tarea_actualizada

            # Se devuelve la tarea actualizada
            return tarea_actualizada

    # Si no se encuentra la tarea, se lanza un error 404
    raise HTTPException(status_code=404, detail="Tarea no encontrada")

# --- RUTA 6: ELIMINAR una tarea (DELETE) ---
# DELETE se emplea para borrar un recurso existente
@app.delete("/tareas/{tarea_id}", status_code=204)
def eliminar_tarea(tarea_id: int):
    # Se recorre la lista con indice
    for indice, tarea in enumerate(tareas_db):
        # Si se encuentra la tarea con el ID correcto...
        if tarea.id == tarea_id:
            # Se elimina de la lista usando pop(indice)
            tareas_db.pop(indice)

            # Se devuelve un mensaje de confirmacion
            # status_code=204 significa "Sin contenido" (exito sin cuerpo de respuesta)
            return {"mensaje": "Tarea eliminada correctamente"}

    # Si no se encuentra la tarea, se lanza un error 404
    raise HTTPException(status_code=404, detail="Tarea no encontrada")
```

Listing 1: Archivo main.py con el CRUD completo de tareas

3. Procedimiento de ejecución

Para poner en funcionamiento la API descrita, se siguen los siguientes pasos:

1. **Instalación de dependencias.** Desde la terminal se instalan los paquetes necesarios:

   ```
   pip install fastapi uvicorn
   ```

2. **Almacenamiento del código.** El contenido del Listado 1 se guarda en un archivo denominado `main.py`.

3. **Ejecución del servidor.** El servidor se inicia mediante el comando:

   ```
   uvicorn main:app --reload
   ```

   En donde cada argumento cumple la siguiente función:

   - `main`: nombre del archivo que contiene la aplicación (sin la extensión `.py`).
   - `app`: nombre del objeto `FastAPI` instanciado en el código.
   - `-reload`: opción que reinicia el servidor automáticamente cada vez que se detectan cambios en el código fuente; resulta útil durante la etapa de desarrollo.

4. **Verificación de la documentación interactiva.** Una vez iniciado el servidor, se accede desde el navegador a la dirección `http://127.0.0.1:8000/docs`, donde FastAPI genera de manera automática una interfaz para probar cada uno de los *endpoints* definidos, sin necesidad de escribir código adicional.

4. Resumen del flujo de comunicación

El siguiente esquema representa el recorrido que sigue una petición desde el cliente hasta obtener una respuesta:

```
Cliente (navegador o aplicación externa)
                ↓
Uvicorn (recibe la petición HTTP y la traduce a ASGI)
                ↓
FastAPI (procesa la lógica y valida los datos)
                ↓
Respuesta en formato JSON enviada de vuelta al cliente
```

En este esquema, Uvicorn desempeña la función de recibir las peticiones y entregarlas al componente que las procesa; FastAPI se encarga de interpretar la lógica y construir las respuestas. La ausencia de cualquiera de los dos impide el funcionamiento completo del sistema: sin Uvicorn, las peticiones no alcanzan a FastAPI; sin FastAPI, Uvicorn no dispone de una aplicación que responder.

Referencias

- FastAPI. *Official Documentation*. https://fastapi.tiangolo.com/
- Uvicorn. *The lightning-fast ASGI server*. https://www.uvicorn.org/
- Pydantic. *Data validation using Python type hints*. https://docs.pydantic.dev/
- MDN Web Docs. *HTTP request methods*. https://developer.mozilla.org/es/docs/Web/HTTP/Methods